

Roteiro do vídeo de apresentação — Projeto 2

Biblioteca

Cada integrante grava **uma parte** separada. O Kalebe (que também apresenta a parte 4) depois junta tudo em um único vídeo. Tempo total alvo: **8 a 12 minutos**.

Antes de começar a gravar (todos leiam)

Preparação uniforme (para os cortes ficarem consistentes)

- **Ferramenta de gravação sugerida:** OBS Studio, Xbox Game Bar (Win+G) ou Loom. Grave em **1920×1080, 30 fps**.
- **Áudio:** fone com microfone se puder. Grave em ambiente silencioso; faça um teste antes.
- **Faça o backend e o frontend estarem rodando** antes de gravar sua parte:

```
docker compose up          # em um terminal, ou:  
# backend: cd backend && npm run dev  
# frontend: cd frontend && npm run dev
```

- **Abas do navegador que devem estar prontas:**
 1. Frontend: <http://localhost:5173>
 2. Swagger: <http://localhost:3000/api-docs>
- **Editor de código aberto** (VS Code) na pasta do projeto — cada um vai precisar abrir arquivos específicos.
- **Faça login antes de gravar** para não perder tempo digitando (a menos que sua parte seja demonstrar o login).

Estrutura do vídeo

Parte	Integrante	Tema	Duração
1	Higor	Abertura, equipe, tema, arquitetura	~2 min
2	Gleisson	Modelagem do banco + API REST + Swagger	~2,5 min
3	Rafael	Autenticação JWT + regras de negócio	~2,5 min
4	Kalebe	Frontend React + demonstração + encerramento	~3 min

Podem trocar quem faz qual parte livremente — só mantenham a ordem para o vídeo final ficar coerente.

Convenção de abertura e encerramento em cada parte

Todos abrem falando seu nome e o que vão apresentar. Todos terminam anunciando quem vem depois. Assim os cortes ficam naturais no vídeo final:

Abertura: *"Olá, eu sou o [seu nome], da disciplina de Programação Web, e nesta parte vou apresentar [tema da parte]."*

Encerramento: *"Agora, o(a) [nome do próximo] vai continuar mostrando [tema da próxima parte]."*

PARTE 1 — Higor (~2 min): Abertura e visão geral

1.1 Falar (roteiro sugerido)

*"Olá professora, meu nome é Higor Silva Fernandes. Junto com Kalebe Silva do Nascimento, Rafael de Godoy Vianna e Gleisson Santos Vieira, apresentamos o Projeto 2 da disciplina: um **Sistema de Gerenciamento de Biblioteca**."*

*"O sistema permite que uma biblioteca controle seu acervo de livros, cadastro de leitores e todo o fluxo de empréstimos e devoluções, com três perfis de acesso diferentes: **Administrador**, **Bibliotecário** e **Leitor**."*

*"A aplicação segue arquitetura cliente-servidor. Do lado do backend, temos uma **API REST em Node.js com Express, Sequelize como ORM, PostgreSQL como banco de dados relacional, autenticação JWT** e documentação com **Swagger**. Do lado do frontend, temos uma **SPA em React** que consome toda a API."*

1.2 Mostrar na tela

- Abra o **README.md** na raiz do projeto e mostre a tabela de integrantes e a seção "Tecnologias utilizadas".
- Abra o VS Code na estrutura do projeto e passe rapidamente pela pasta **backend/** e **frontend/** (só para dar a visão geral do monorepo).
- Volte pro navegador em <http://localhost:5173> mostrando a tela de login.

1.3 Encerramento

"Agora o Gleisson vai explicar como modelamos o banco de dados e como estruturamos os endpoints da API REST."

PARTE 2 — Gleisson (~2,5 min): Modelagem + API REST + Swagger

2.1 Falar

*"Boa tarde, sou Gleisson Santos Vieira. Vou apresentar a **modelagem do banco** e a **API REST**."*

*"Temos quatro tabelas principais: **users** (que guarda o login e o papel de cada pessoa do sistema), **readers** (o perfil de leitor, com CPF ou RA, telefone, endereço e status ativo/inativo — relacionado 1:1 com users), **books** (com título, autor, editora, ano, categoria, ISBN e as quantidades total e disponível) e **loans**, que amarra um livro a um leitor com data de empréstimo, data prevista de devolução e status: aberto, devolvido ou atrasado."*

*"Tudo foi feito com **migrations do Sequelize**. Também disponibilizamos um **script.sql** puro na pasta **backend/database/**, para o caso de a professora preferir criar o schema sem o Sequelize CLI."*

*"A API segue o padrão REST com verbos GET, POST, PUT, DELETE e PATCH. Toda a documentação está disponível em **/api-docs** através do **Swagger**."*

2.2 Mostrar na tela

- Abra **backend/src/models/loan.js** no VS Code e destaque as associações **belongsTo(Reader)** e **belongsTo(Book)**.
- Abra **backend/database/script.sql** e passe rapidamente pelas 4 tabelas com FKs.
- Abra **<http://localhost:3000/api-docs>** no navegador. Mostre as seções:
- **Autenticação** (rota **POST /api/auth/login**)
- **Livros** (GET com filtros de título/autor/categoria/ISBN/disponibilidade, POST, PUT, DELETE)
- **Leitores**
- **Empréstimos**
- Expanda **POST /api/books**, mostre o **requestBody** com os campos, expanda **GET /api/books** e mostre os parâmetros de filtro.

2.3 Encerramento

"O Rafael agora vai explicar a autenticação com JWT e as regras de negócio dos empréstimos."

PARTE 3 — Rafael (~2,5 min): JWT + RBAC + regras de negócio

3.1 Falar

"Oi, meu nome é Rafael de Godoy Vianna. Vou apresentar a **autenticação com JWT** e as **regras de negócio críticas** do sistema."

"Quando o usuário faz login, o backend valida a senha usando **bcrypt** — a senha nunca fica salva em texto puro. Se as credenciais estão certas, ele gera um **token JWT** assinado com uma chave secreta, contendo o id, e-mail e papel do usuário. Esse token vai no header **Authorization: Bearer** de todas as próximas requisições."

"Duas camadas de middleware protegem a API: uma que verifica se o token é válido (**authenticate**), e outra que checa o papel do usuário (**authorize**). Isso é o que chamamos de RBAC — controle de acesso por papel. O Administrador pode tudo. O Bibliotecário pode gerenciar livros, leitores e empréstimos, mas não usuários do sistema. O Leitor só visualiza livros e os próprios empréstimos."

"Sobre as regras de negócio dos empréstimos: ao registrar um empréstimo, o sistema **verifica se há exemplares disponíveis e se o leitor está ativo** — se falhar, retorna erro. Se passar, decrementa a quantidade disponível dentro de uma transação. Na devolução, incrementa de volta e marca a data. E empréstimos com data prevista vencida são automaticamente marcados como atrasados quando alguém consulta."

3.2 Mostrar na tela

- Abra `backend/src/controllers/authController.js` e destaque a chamada `bcrypt.compare` e `signToken`.
- Abra `backend/src/middlewares/auth.js` e mostre as funções `authenticate` e `authorize`.
- Abra `backend/src/routes/bookRoutes.js` e destaque uma linha como:

```
router.post('/', authorize('administrador', 'bibliotecario'), create);
```

- Abra `backend/src/controllers/loanController.js` e destaque:
 - a checagem `if (reader.status !== 'ativo')`
 - a checagem `if (book.availableQuantity < 1)`
 - o `book.availableQuantity -= 1` dentro da transação
 - o trecho de `markOverdueLoans`

3.3 Encerramento

"Para fechar, o Kalebe vai mostrar o frontend em React e fazer uma demonstração completa do sistema funcionando."

PARTE 4 — Kalebe (~3 min): Frontend + demonstração + encerramento

4.1 Falar

*"Fala pessoal, sou Kalebe Silva do Nascimento. Vou apresentar o **frontend em React** e fazer uma demonstração completa do fluxo do sistema."*

*"O frontend é uma SPA feita com **React** e **Vite**. Usamos **react-router-dom** para navegação, **axios** com um interceptor que injeta o token JWT automaticamente em toda requisição, e um **AuthContext** para gerenciar o estado de login globalmente. As rotas privadas são protegidas por um **ProtectedRoute** que também controla acesso por papel."*

4.2 Demonstração ao vivo (siga a ordem para mostrar tudo)

"Vamos ver o sistema funcionando com os três perfis."

Como Bibliotecário:

1. Faça login com **bibliotecario@biblioteca.com** / **senha123**.
2. Mostre o **Dashboard** — comente os KPIs (livros cadastrados, exemplares disponíveis, leitores ativos, empréstimos por status) e o gráfico + top 5 livros.
3. Vá em **Livros**. Mostre a listagem, use o filtro por título ou categoria, e clique em "Novo livro" para mostrar o formulário (pode cancelar sem salvar).
4. Vá em **Leitores** e mostre a listagem/busca.
5. Vá em **Empréstimos**, clique em "Novo empréstimo", selecione um leitor e um livro, e registre.
6. Volte à aba **Livros** — mostre que a quantidade disponível daquele livro caiu.
7. Volte em **Empréstimos** e registre a devolução clicando em "Devolver".
8. Mostre que a quantidade voltou.

Como Leitor:

9. Faça logout, entre com **leitor1@biblioteca.com** / **senha123**.
10. Mostre que o **menu ficou reduzido** (sem "Leitores", sem "Usuários", sem "Dashboard") — comente que isso é o RBAC funcionando.
11. Vá em **Empréstimos** e mostre que ele só enxerga os próprios empréstimos.

Como Administrador (rapidinho):

12. Faça logout, entre com **admin@biblioteca.com** / **senha123**.
13. Mostre que agora aparece o menu **Usuários**, e abra a tela mostrando que dá para cadastrar/editar usuários e definir o papel de cada um.

4.3 Fechamento

*"Além do obrigatório, implementamos quatro bônus: **Dashboard com gráficos, paginação nas consultas, testes automatizados com Jest e Supertest** e **dockerização completa** — dá para*

subir todo o projeto com um único `docker compose up`."

*"Todo o código, instruções de execução, credenciais e explicações estão no README.
Obrigado pela atenção!"*

Dicas finais para o Kalebe (juntar os vídeos)

- Ferramenta sugerida: **Clipchamp** (já vem no Windows 11), **CapCut** ou **DaVinci Resolve** (gratuito e mais completo).
 - Ordem: Parte 1 → Parte 2 → Parte 3 → Parte 4.
 - Faça uma transição simples de fade (0,3 s) entre cada parte para não ficar "cortado" demais.
 - Se alguma parte ficou muito longa, dá pra cortar pausas ou trechos onde ninguém tá falando.
 - Ao final, exporte em **1080p H.264**, MP4.
 - Sugestão de nome do arquivo: `projeto2-biblioteca-apresentacao.mp4`.
 - Suba no YouTube como **"Não listado"** (ninguém acha por busca, mas quem tem o link assiste) e cole o link no README antes de gerar o ZIP final.
-

Checklist de gravação (imprimir e marcar)

Antes de cada um começar a gravar sua parte:

- ☐ Backend está rodando em `localhost:3000`
- ☐ Frontend está rodando em `localhost:5173`
- ☐ Banco tem os seeds populados (usuários e livros)
- ☐ Aba do navegador aberta na página que vai começar
- ☐ Editor de código aberto nos arquivos que vai mostrar
- ☐ Notificações do sistema silenciadas (Discord, WhatsApp, etc.)
- ☐ Ambiente silencioso, microfone testado
- ☐ Aumentei o zoom do VS Code (Ctrl + "+" umas 3 vezes) para o código ficar legível
- ☐ Aumentei o zoom do navegador (Ctrl + "+") para a UI ficar legível